

Java Interview Series · Part 6 · Q255–Q304

Top Java Coding Questions · Round 2

50 harder coding problems: two pointers, sliding windows, monotonic stacks, heaps, trees, graphs, dynamic programming and backtracking, each run and verified on a real JDK.

50	50	16	17–25
questions	code examples	diagrams	Java LTS

How to use this guide

Every question follows the same layout so you can revise fast the night before and go deep when you have time. Read the **Quick answer** first. If you can say it in your own words, move on. If not, read the explanation and run the code.

Q07 `TC-007`

Number and ID. Q-numbers run continuously across the whole TechTribers Java series, so every question has one unique number. The topic ID stays fixed across editions.

• Basic • Intermediate • Advanced

Difficulty. Basic for 0–2 years, Intermediate for 2–5 years, Advanced for senior rounds.

⚡ Quick answer

The one-line answer. Lead with this in the interview, then expand.

⚠️ **Interview trap**

A common wrong answer or an edge case interviewers use to filter candidates.

💡 **Pro tip**

What a senior engineer would add: production practice, design reasoning, modern Java.

🎯 **Likely follow-up**

The next question the interviewer is likely to ask, with the answer.

50	8	50	16
questions	sections	code examples	diagrams

Examples target modern Java (17, 21 and 25 LTS). Where a feature needs a newer version, the answer says which one.

© TechTribers. For personal use only. Do not redistribute.

Contents

🧮	01	Arrays, Two Pointers & Prefix Sums	Q255–Q264	5
🔤	02	Strings & Sliding Window	Q265–Q271	13
📚	03	Stacks, Queues & Heaps	Q272–Q277	19
🔗	04	Linked Lists, Advanced	Q278–Q280	25
🌳	05	Trees & Tries	Q281–Q287	28
🕸️	06	Graphs	Q288–Q293	34
🧠	07	Dynamic Programming	Q294–Q301	40
♟️	08	Backtracking	Q302–Q304	46

© TechTribers. For personal use only. Do not redistribute.

All questions

🧮 Arrays, Two Pointers & Prefix Sums

Q255 3Sum: find all unique triplets that sum to zero. 5
Q256 Container with most water. 6
Q257 Trapping rain water. 7
Q258 Find the length of the longest consecutive sequence in an unsorted array in O(n). 8
Q259 Count subarrays whose sum equals K. 8
Q260 Find the majority element (appears more than n/2 times) in O(1) space. 9
Q261 Find the next lexicographic permutation in place. 10
Q262 Sort an array of 0s, 1s and 2s in one pass (Dutch national flag). 10
Q263 Rotate an n × n matrix by 90° clockwise in place. 11
Q264 Print a matrix in spiral order. 12

🔤 Strings & Sliding Window

Q265 Minimum window substring containing all characters of a pattern. 13
Q266 Longest substring with the same letter after at most K replacements. 14
Q267 Group anagrams together. 15
Q268 Find all start indices of a pattern's anagrams in a string. 15
Q269 Implement `atoi` : convert a string to a 32-bit integer. 16
Q270 Find a pattern in text in linear time (KMP algorithm). 17
Q271 Convert Roman numerals to integers and back. 18

📚 Stacks, Queues & Heaps

Q272 Find the next greater element for every element in an array. 19
Q273 Find the largest rectangle in a histogram. 20
Q274 Find the maximum in every sliding window of size k. 21
Q275 Merge K sorted linked lists. 22
Q276 Find the median of a data stream. 23
Q277 Implement a min-heap from scratch. 24

🔗 Linked Lists, Advanced

Q278 Check if a linked list is a palindrome in O(1) extra space. 25
Q279 Add two numbers stored as reversed-digit linked lists. 26
Q280 Sort a linked list in O(n log n) time. 27

🌳 Trees & Tries

Q281 Find the diameter of a binary tree. 28
Q282 Find the maximum path sum in a binary tree. 29
Q283 Serialize and deserialize a binary tree. 29
Q284 Find the k-th smallest element in a BST. 30
Q285 Construct a binary tree from preorder and inorder traversals. 31
Q286 Invert (mirror) a binary tree. 32
Q287 Implement a Trie (prefix tree) with insert, search and startsWith. 33

🕸️ Graphs

Q288 Rotting oranges: minimum minutes until all oranges rot (multi-source BFS). 34
Q289 Word ladder: shortest transformation sequence from one word to another. 35
Q290 Course schedule: topological sort and cycle detection in a directed graph. 36
Q291 Find shortest paths with Dijkstra's algorithm. 37
Q292 Implement Union-Find and count connected components. 38
Q293 Check whether a graph is bipartite (two-colourable). 39

© TechTribers. For personal use only. Do not redistribute.

🧠 Dynamic Programming

Q294 House robber: maximum sum of non-adjacent elements. 40

Q295 Longest increasing subsequence in O(n log n). 41

Q296 0/1 knapsack: maximise value within a weight capacity. 42

Q297 Edit distance (Levenshtein) between two strings. 43

Q298 Word break: can a string be segmented into dictionary words? 44

Q299 Count unique paths in a grid (with and without obstacles). 44

Q300 Decode ways: count decodings of a digit string (A=1 … Z=26). 45

Q301 Maximum product subarray. 45

♟ Backtracking

Q302 Generate all subsets (power set) of a set. 46

Q303 Combination sum: find all combinations that add up to a target (reuse allowed). 47

Q304 Solve the N-Queens problem. 48

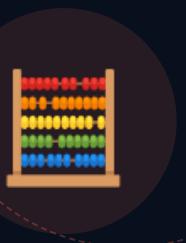
© TechTribers. For personal use only. Do not redistribute.

Section 01

Arrays, Two Pointers & Prefix Sums

The array problems that separate a pass from a strong hire: sorting plus two pointers, prefix sums, and in-place tricks.

Questions 255–264 · 10 questions



3️⃣ Q255 TC-051

• Intermediate

3Sum: find all unique triplets that sum to zero.

⚡ Quick answer

Sort, fix each element `i`, then use two pointers on the rest to find pairs summing to `-nums[i]`. Skip equal neighbours to avoid duplicate triplets. O(n²) time, O(1) extra space besides the output.

Java

```
static List<List<Integer>> threeSum(int[] a) {
    int[] nums = a.clone();
    Arrays.sort(nums);
    List<List<Integer>> out = new ArrayList<>();
    for (int i = 0; i < nums.length - 2 && nums[i] <= 0; i++) {
        if (i > 0 && nums[i] == nums[i - 1]) continue;              // skip duplicate anchors
        int l = i + 1, r = nums.length - 1;
        while (l < r) {
            int sum = nums[i] + nums[l] + nums[r];
            if (sum < 0) l++;
            else if (sum > 0) r--;
            else {
                out.add(List.of(nums[i], nums[l], nums[r]));
                while (l < r && nums[l] == nums[l + 1]) l++;
                while (l < r && nums[r] == nums[r - 1]) r--;
                l++; r--;
            }
        }
    }
    return out;
}
System.out.println(threeSum(new int[]{-1, 0, 1, 2, -1, -4}) + " " + threeSum(new int[]{0, 0, 0,
0}));
```

▶ Output

```
[[-1, -1, 2], [-1, 0, 1]] [[0, 0, 0]]
```


© TechTribers. For personal use only. Do not redistribute.

💡 **Pro tip**

The `nums[i] <= 0` early stop works because the array is sorted: once the anchor is positive, no triplet can sum to zero.

Q256 TC-052 • Intermediate

Container with most water.

⚡ **Quick answer**

Start pointers at both ends; the area is `min(h[l], h[r]) × (r - l)`. Always move the shorter side inward, because moving the taller one can never increase the area. O(n) time, O(1) space.

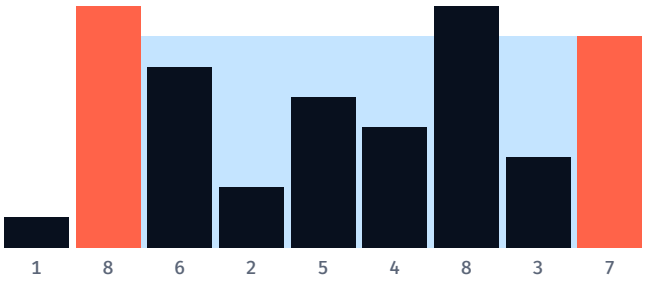
Java

```
static int maxArea(int[] h) {
    int l = 0, r = h.length - 1, best = 0;
    while (l < r) {
        best = Math.max(best, Math.min(h[l], h[r]) * (r - l));
        if (h[l] < h[r]) l++; else r--;
    }
    return best;
}
System.out.println(maxArea(new int[]{1, 8, 6, 2, 5, 4, 8, 3, 7}) + " " + maxArea(new int[]{1, 1}));
```

▶ **Output**

```
49 1
```

Container with most water: area = min(h[l], h[r]) × (r – l); move the shorter side



best: l=1, r=8
min(8, 7) × 7 = 49

© TechTribers. For personal use only. Do not redistribute.

Q257 TC-053

• Advanced

Trapping rain water.

⚡ Quick answer

Water above bar i is `min(maxLeft, maxRight) - height[i]`. With two pointers, process the side whose running max is smaller, because that side's water level is already decided. O(n) time, O(1) space.

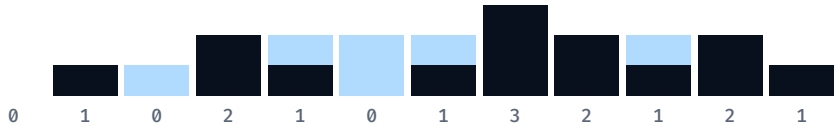
Java

```
static int trap(int[] h) {
    int l = 0, r = h.length - 1, leftMax = 0, rightMax = 0, water = 0;
    while (l < r) {
        if (h[l] < h[r]) {
            leftMax = Math.max(leftMax, h[l]);
            water += leftMax - h[l++];
        } else {
            rightMax = Math.max(rightMax, h[r]);
            water += rightMax - h[r--];
        }
    }
    return water;
}
System.out.println(trap(new int[]{0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1}) + " " + trap(new int[]{4, 2, 0, 3, 2, 5}));
```

▶ Output

```
6 9
```

Trapped water = Σ min(maxLeft, maxRight) − height → 6 units



two pointers: move the side with the lower max

O(n) time · O(1) space

💡 Pro tip

Explain it in three steps: brute force O(n²) → precomputed left/right max arrays O(n) space → two pointers O(1) space. Interviewers grade the progression, not just the final code.

© TechTribers. For personal use only. Do not redistribute.

Q258 TC-054 • Intermediate

Find the length of the longest consecutive sequence in an unsorted array in O(n).

⚡ Quick answer

Put all numbers in a `HashSet`. Only start counting from numbers that begin a sequence (`x - 1` not in the set), then extend while `x + 1` exists. Each number is visited at most twice: O(n).

Java

```
static int longestConsecutive(int[] nums) {
    Set<Integer> set = new HashSet<>();
    for (int x : nums) set.add(x);
    int best = 0;
    for (int x : set) {
        if (set.contains(x - 1)) continue;          // not a sequence start
        int len = 1;
        while (set.contains(x + len)) len++;
        best = Math.max(best, len);
    }
    return best;
}
System.out.println(longestConsecutive(new int[]{100, 4, 200, 1, 3, 2}) + " " +
longestConsecutive(new int[]{0, 3, 7, 2, 5, 8, 4, 6, 0, 1}));
```

▶ Output

```
4 9
```

⚠️ Interview trap

Sorting first is O(n log n), which fails the "O(n)" requirement. Starting a count from every element (not only sequence starts) degrades to O(n²).

Q259 TC-055 • Intermediate

Count subarrays whose sum equals K.

⚡ Quick answer

Keep a running prefix sum and a map from prefix value to how many times it occurred. At each index, add `count[prefix - k]` to the answer. O(n) time, O(n) space, and it handles negative numbers.

Java

```
static int subarraySum(int[] nums, int k) {
    Map<Integer, Integer> seen = new HashMap<>(Map.of(0, 1));      // empty prefix
    int prefix = 0, count = 0;
    for (int x : nums) {
```


© TechTribers. For personal use only. Do not redistribute.

```
        prefix += x;
        count += seen.getOrDefault(prefix - k, 0);
        seen.merge(prefix, 1, Integer::sum);
    }
    return count;
}
System.out.println(subarraySum(new int[]{1, 1, 1}, 2) + " " + subarraySum(new int[]{1, 2, 3, -2, 5},
3) + " " + subarraySum(new int[]{3, -3, 3}, 3));
```

▶ Output

```
2 4 3
```

Subarray sum = k (k = 3): count earlier prefixes equal to prefix − k



map {prefix → count}

```
prefix 3 - 3 = 0 seen → [1,2]
prefix 6 - 3 = 3 seen → [3]
prefix 4 - 3 = 1 seen → [2,3,-2]
prefix 9 - 3 = 6 seen → [-2,5] → 4
```

one pass · O(n) time · O(n) space · works with negatives (sliding window doesn't)

⚠ **Interview trap**

A sliding window only works when all numbers are non-negative. With negatives, the window sum isn't monotonic, so prefix sums are required.

Q260 TC-056 • Intermediate

Find the majority element (appears more than n/2 times) in O(1) space.

⚡ **Quick answer**

Boyer–Moore voting: keep a candidate and a counter; a matching element increments, a different one decrements, and at zero you switch candidates. O(n) time, O(1) space. Verify with a second pass if a majority isn't guaranteed.

Java

```
static OptionalInt majority(int[] a) {
    int candidate = 0, count = 0;
    for (int x : a) {
        if (count == 0) candidate = x;
        count += x == candidate ? 1 : -1;
    }
    int c = candidate;
    long occurrences = Arrays.stream(a).filter(x -> x == c).count();     // verification pass
    return occurrences > a.length / 2 ? OptionalInt.of(c) : OptionalInt.empty();
```


© TechTribers. For personal use only. Do not redistribute.

```
}
System.out.println(majority(new int[]{2, 2, 1, 1, 1, 2, 2}) + " " + majority(new int[]{1, 2, 3}));
```

▶ Output

```
OptionalInt[2] OptionalInt.empty
```

⏭ Q261 TC-057 • Advanced

Find the next lexicographic permutation in place.

⚡ Quick answer

From the right, find the first index `i` with `a[i] < a[i+1]`; find the rightmost `j` with `a[j] > a[i]`; swap them; reverse the suffix after `i`. If no such `i` exists, reverse the whole array. O(n) time, O(1) space.

Java

```
static void nextPermutation(int[] a) {
    int i = a.length - 2;
    while (i >= 0 && a[i] >= a[i + 1]) i--;
    if (i >= 0) {
        int j = a.length - 1;
        while (a[j] <= a[i]) j--;
        int t = a[i]; a[i] = a[j]; a[j] = t;
    }
    for (int l = i + 1, r = a.length - 1; l < r; l++, r--) { int t = a[l]; a[l] = a[r]; a[r] = t; }
}
for (int[] a : new int[][]{{1, 2, 3}, {3, 2, 1}, {1, 1, 5}, {1, 3, 2}}) {
    nextPermutation(a);
    System.out.print(Arrays.toString(a) + " ");
}
```

▶ Output

```
[1, 3, 2] [1, 2, 3] [1, 5, 1] [2, 1, 3]
```

🇳🇱 Q262 TC-058 • Intermediate

Sort an array of 0s, 1s and 2s in one pass (Dutch national flag).

⚡ Quick answer

Three pointers: `low` (next 0 slot), `mid` (current), `high` (next 2 slot). A 0 swaps with `low` and advances both; a 1 advances `mid`; a 2 swaps with `high` and decrements it without advancing `mid`. O(n), one pass, O(1) space.

© TechTribers. For personal use only. Do not redistribute.

```java
static void sortColors(int[] a) {
    int low = 0, mid = 0, high = a.length - 1;
    while (mid <= high) {
        switch (a[mid]) {
            case 0 -> { int t = a[low]; a[low++] = a[mid]; a[mid++] = t; }
            case 1 -> mid++;
            default -> { int t = a[high]; a[high--] = a[mid]; a[mid] = t; }     // don't advance mid
        }
    }
}
int[] a = {2, 0, 2, 1, 1, 0, 1, 2, 0};
sortColors(a);
System.out.println(Arrays.toString(a));
```

▶ Output

```
[0, 0, 0, 1, 1, 1, 2, 2, 2]
```

Dutch national flag: three regions maintained by low, mid, high



[0..low) = 0s · [low..mid) = 1s · [mid..high] = unknown · (high..end] = 2s → one pass, O(1) space

⚠️ **Interview trap**

After swapping with `high`, the element now at `mid` hasn't been examined yet. Incrementing `mid` there is the classic bug.

Q263 TC-059 • Intermediate

Rotate an n × n matrix by 90° clockwise in place.

⚡ **Quick answer**

Transpose (swap `m[i][j]` with `m[j][i]`), then reverse each row. O(n²) time, O(1) space. For counter-clockwise, reverse each row first and then transpose.

```java
static void rotate(int[][] m) {
    int n = m.length;
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++) { int t = m[i][j]; m[i][j] = m[j][i]; m[j][i] = t; }
    for (int[] row : m)
        for (int l = 0, r = n - 1; l < r; l++, r--) { int t = row[l]; row[l] = row[r]; row[r] = t; }
```


© TechTribers. For personal use only. Do not redistribute.

```
}
int[][] m = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
rotate(m);
System.out.println(Arrays.deepToString(m));
```

▶ Output

```
[[7, 4, 1], [8, 5, 2], [9, 6, 3]]
```

Q264 TC-060 • Intermediate

Print a matrix in spiral order.

⚡ Quick answer

Maintain four boundaries (`top` , `bottom` , `left` , `right`); walk right, down, left, up, shrinking the matching boundary after each side, and stop when they cross. O(m × n).

Java

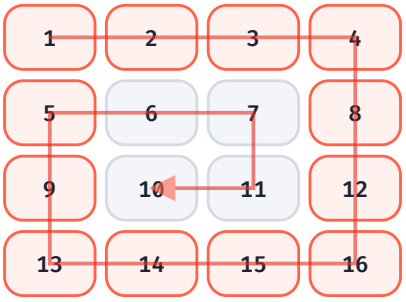
```
static List<Integer> spiral(int[][] m) {
    List<Integer> out = new ArrayList<>();
    int top = 0, bottom = m.length - 1, left = 0, right = m[0].length - 1;
    while (top <= bottom && left <= right) {
        for (int j = left; j <= right; j++) out.add(m[top][j]);
        top++;
        for (int i = top; i <= bottom; i++) out.add(m[i][right]);
        right--;
        if (top <= bottom) { for (int j = right; j >= left; j--) out.add(m[bottom][j]); bottom--; }
        if (left <= right) { for (int i = bottom; i >= top; i--) out.add(m[i][left]); left++; }
    }
    return out;
}
System.out.println(spiral(new int[][]{{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}, {13, 14, 15,
16}}));
System.out.println(spiral(new int[][]{{1, 2, 3}, {4, 5, 6}}));
```

▶ Output

```
[1, 2, 3, 4, 8, 12, 16, 15, 14, 13, 9, 5, 6, 7, 11, 10]
[1, 2, 3, 6, 5, 4]
```


© TechTribers. For personal use only. Do not redistribute.

Spiral order: shrink four boundaries (top, bottom, left, right) after each side



`1 2 3 4 8 12 16 15 14 13 9 5 6 7 11 10`

outer ring (shaded) then inner ring

⚠️ **Interview trap**

Without the `top <= bottom` and `left <= right` checks before the last two sides, non-square matrices print elements twice.

Section 02

Strings & Sliding Window

Harder string problems: variable windows, anagram grouping, parsing with overflow rules and linear-time pattern matching.

Questions 265–271 · 7 questions

Q265 TC-061

• Advanced

Minimum window substring containing all characters of a pattern.

⚡ **Quick answer**

Count the pattern's characters; expand `right` and decrement `missing` when a needed character enters; while nothing is missing, record the window and shrink from `left`. O(|s| + |t|) time.

Java

```
static String minWindow(String s, String t) {
    int[] need = new int[128];
    for (char c : t.toCharArray()) need[c]++;
    int missing = t.length(), left = 0, bestStart = 0, bestLen = Integer.MAX_VALUE;
    for (int right = 0; right < s.length(); right++) {
        if (need[s.charAt(right)]-- > 0) missing--;
        while (missing == 0) {
            if (right - left + 1 < bestLen) { bestLen = right - left + 1; bestStart = left; }
            if (++need[s.charAt(left++)] > 0) missing++;              // a needed char leaves
        }
```


© TechTribers. For personal use only. Do not redistribute.

```
    }
    return bestLen == Integer.MAX_VALUE ? "" : s.substring(bestStart, bestStart + bestLen);
}
System.out.println(minWindow("ADOBECODEBANC", "ABC") + " | " + minWindow("a", "aa") + " | " +
minWindow("aa", "aa"));
```

▶ Output

```
BANC |  | aa
```

Q266 TC-062 • Advanced

Longest substring with the same letter after at most K replacements.

⚡ Quick answer

Sliding window tracking the count of the most frequent character in the window. The window is valid while `length - maxFreq ≤ k`; otherwise move `left`. O(n) time, O(26) space.

Java

```
static int characterReplacement(String s, int k) {
    int[] count = new int[26];
    int left = 0, maxFreq = 0, best = 0;
    for (int right = 0; right < s.length(); right++) {
        maxFreq = Math.max(maxFreq, ++count[s.charAt(right) - 'A']);
        while (right - left + 1 - maxFreq > k) count[s.charAt(left++) - 'A']--;
        best = Math.max(best, right - left + 1);
    }
    return best;
}
System.out.println(characterReplacement("ABAB", 2) + " " + characterReplacement("AABABBA", 1));
```

▶ Output

```
4 4
```

💡 Pro tip

`maxFreq` is never decreased when the window shrinks. That's safe: the answer only improves when a larger `maxFreq` appears, so a stale value can't produce a wrong maximum.

© TechTribers. For personal use only. Do not redistribute.

Q267 TC-063

• Intermediate

Group anagrams together.

⚡ Quick answer

Use a canonical key per word, either the sorted characters or a 26-letter count signature, and group words by key in a `HashMap`. O(n · k log k) with sorting or O(n · k) with counts, for n words of length k.

Java

```
static Collection<List<String>> groupAnagrams(List<String> words) {
    Map<String, List<String>> groups = new TreeMap<>();
    for (String w : words) {
        int[] count = new int[26];
        for (char c : w.toCharArray()) count[c - 'a']++;
        groups.computeIfAbsent(Arrays.toString(count), k -> new ArrayList<>()).add(w);
    }
    return groups.values();
}
List<List<String>> g = new ArrayList<>(groupAnagrams(List.of("eat", "tea", "tan", "ate", "nat",
"bat")));
g.sort(Comparator.comparing(l -> l.get(0)));
System.out.println(g);
```

▶ Output

```
[[bat], [eat, tea, ate], [tan, nat]]
```

Q268 TC-064

• Intermediate

Find all start indices of a pattern's anagrams in a string.

⚡ Quick answer

Fixed-size sliding window of length |p| with character counts. Slide one step at a time, adding the new character and removing the old one, and compare counts (or track a match counter). O(n · 26) or O(n).

Java

```
static List<Integer> findAnagrams(String s, String p) {
    List<Integer> out = new ArrayList<>();
    if (p.length() > s.length()) return out;
    int[] need = new int[26], window = new int[26];
    for (char c : p.toCharArray()) need[c - 'a']++;
    for (int i = 0; i < s.length(); i++) {
        window[s.charAt(i) - 'a']++;
        if (i >= p.length()) window[s.charAt(i - p.length()) - 'a']--;
        if (i >= p.length() - 1 && Arrays.equals(need, window)) out.add(i - p.length() + 1);
    }
    return out;
```


© TechTribers. For personal use only. Do not redistribute.

```
}
System.out.println(findAnagrams("cbaebabacd", "abc") + " " + findAnagrams("abab", "ab"));
```

▶ Output

```
[0, 6] [0, 1, 2]
```

🔢 Q269 TC-065 • Intermediate

Implement `atoi` : convert a string to a 32-bit integer.

⚡ Quick answer

Skip leading spaces, read an optional sign, then accumulate digits until a non-digit, clamping to `Integer.MIN_VALUE` / `MAX_VALUE` before overflow. O(n). The value is in the edge cases, not the loop.

Java

```
static int atoi(String s) {
    int i = 0, n = s.length(), sign = 1;
    long result = 0;
    while (i < n && s.charAt(i) == ' ') i++;
    if (i < n && (s.charAt(i) == '+' || s.charAt(i) == '-')) sign = s.charAt(i++) == '-' ? -1 : 1;
    while (i < n && Character.isDigit(s.charAt(i))) {
        result = result * 10 + (s.charAt(i++) - '0');
        if (result * sign > Integer.MAX_VALUE) return Integer.MAX_VALUE;
        if (result * sign < Integer.MIN_VALUE) return Integer.MIN_VALUE;
    }
    return (int) (result * sign);
}
for (String s : List.of("42", "   -42", "4193 with words", "words 987", "-91283472332", "+1", ""))
    System.out.print(atoi(s) + " ");
```

▶ Output

```
42 -42 4193 0 -2147483648 1 0
```

⚠️ Interview trap

`Integer.parseInt` throws on "4193 with words" and on overflow; `atoi` must stop at the first non-digit and clamp. Clarify the expected behaviour before coding.

© TechTribers. For personal use only. Do not redistribute.

Q270 TC-066

Advanced

Find a pattern in text in linear time (KMP algorithm).

⚡ Quick answer

Precompute the LPS array (longest proper prefix that is also a suffix for each pattern prefix). On a mismatch, jump the pattern pointer to `lps[j - 1]` instead of restarting, so the text pointer never moves back. O(n + m).

Java

```
static int[] lps(String p) {
    int[] lps = new int[p.length()];
    for (int i = 1, len = 0; i < p.length(); ) {
        if (p.charAt(i) == p.charAt(len)) lps[i++] = ++len;
        else if (len > 0) len = lps[len - 1];
        else lps[i++] = 0;
    }
    return lps;
}
static List<Integer> kmp(String text, String p) {
    List<Integer> hits = new ArrayList<>();
    int[] lps = lps(p);
    for (int i = 0, j = 0; i < text.length(); ) {
        if (text.charAt(i) == p.charAt(j)) { i++; j++; if (j == p.length()) { hits.add(i - j); j =
lps[j - 1]; } }
        else if (j > 0) j = lps[j - 1];
        else i++;
    }
    return hits;
}
System.out.println(Arrays.toString(lps("ABABCABAB")) + " " + kmp("ABABDABACDABABCABABABABCABAB",
"ABABCABAB"));
```

▶ Output

```
[0, 0, 1, 2, 0, 1, 2, 3, 4] [10, 19]
```

💡 Pro tip

`String.indexOf` uses a simple scan (fast in practice for short patterns). KMP matters for streaming text, long repetitive patterns, and as the basis for Aho-Corasick.

© TechTribers. For personal use only. Do not redistribute.

Q271 TC-067

• Basic

Convert Roman numerals to integers and back.

⚡ Quick answer

Roman → int: add each symbol's value, but subtract it when a larger symbol follows (IV, IX, XL…). Int → Roman: greedily subtract the largest value from a 13-entry table including the subtractive pairs. Both O(1) for the valid range 1–3999.

Java

```
static int romanToInt(String s) {
    Map<Character, Integer> v = Map.of('I', 1, 'V', 5, 'X', 10, 'L', 50, 'C', 100, 'D', 500, 'M',
1000);
    int total = 0;
    for (int i = 0; i < s.length(); i++) {
        int cur = v.get(s.charAt(i));
        total += i + 1 < s.length() && cur < v.get(s.charAt(i + 1)) ? -cur : cur;
    }
    return total;
}
static String intToRoman(int n) {
    int[] vals = {1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1};
    String[] syms = {"M", "CM", "D", "CD", "C", "XC", "L", "XL", "X", "IX", "V", "IV", "I"};
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < vals.length; i++) while (n >= vals[i]) { n -= vals[i]; sb.append(syms[i]); }
    return sb.toString();
}
System.out.println(romanToInt("MCMXCIV") + " " + romanToInt("LVIII") + " " + intToRoman(2026) + " "
+ intToRoman(3999));
```

▶ Output

```
1994 58 MMXXVI MMMCMXCIX
```


© TechTribers. For personal use only. Do not redistribute.

Section 03

Stacks, Queues & Heaps

Monotonic stacks, deques and heaps: the data structures that turn O(n²) scans into O(n) or O(n log k).

Questions 272–277 · 6 questions



➡️ Q272 TC-068

• Intermediate

Find the next greater element for every element in an array.

⚡ Quick answer

Scan left to right with a stack of indices whose answer is still unknown (values decreasing). When the current value is larger than the top, it is the top's next greater element; pop and record. O(n): each index is pushed and popped once.

Java

```
static int[] nextGreater(int[] a) {
    int[] res = new int[a.length];
    Arrays.fill(res, -1);
    Deque<Integer> stack = new ArrayDeque<>();
    for (int i = 0; i < a.length; i++) {
        while (!stack.isEmpty() && a[i] > a[stack.peek()]) res[stack.pop()] = a[i];
        stack.push(i);
    }
    return res;
}
static int[] dailyTemperatures(int[] t) {                // same pattern: days to wait
    int[] res = new int[t.length];
    Deque<Integer> stack = new ArrayDeque<>();
    for (int i = 0; i < t.length; i++) {
        while (!stack.isEmpty() && t[i] > t[stack.peek()]) { int j = stack.pop(); res[j] = i - j; }
        stack.push(i);
    }
    return res;
}
System.out.println(Arrays.toString(nextGreater(new int[]{2, 1, 2, 4, 3})) + " " +
Arrays.toString(dailyTemperatures(new int[]{73, 74, 75, 71, 69, 72, 76, 73})));
```

▶ Output

```
[4, 2, 4, -1, -1] [1, 1, 4, 2, 1, 1, 0, 0]
```


© TechTribers. For personal use only. Do not redistribute.



Q273 TC-069 • Advanced

Find the largest rectangle in a histogram.

⚡ **Quick answer**

Keep a stack of bar indices with increasing heights. When a lower bar arrives, pop: the popped bar's rectangle extends from the new stack top + 1 to the current index − 1. Process a sentinel 0 at the end. O(n).

Java

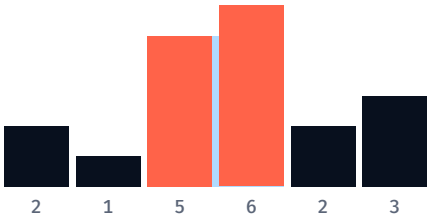
```
static int largestRectangle(int[] h) {
    Deque<Integer> stack = new ArrayDeque<>();
    int best = 0;
    for (int i = 0; i <= h.length; i++) {
        int cur = i == h.length ? 0 : h[i];                    // sentinel flushes the stack
        while (!stack.isEmpty() && cur < h[stack.peek()]) {
            int height = h[stack.pop()];
            int width = stack.isEmpty() ? i : i - stack.peek() - 1;
            best = Math.max(best, height * width);
        }
        stack.push(i);
    }
    return best;
}
System.out.println(largestRectangle(new int[]{2, 1, 5, 6, 2, 3}) + " " + largestRectangle(new int[]
{2, 4}) + " " + largestRectangle(new int[]{3, 3, 3}));
```

▶ Output

```
10 4 9
```


© TechTribers. For personal use only. Do not redistribute.

Largest rectangle in histogram: monotonic stack finds each bar's left/right limits → 10



height 5 × width 2 = 10

pop a bar when a lower one arrives:
`width = i - stack.peek() - 1`

🎯 **Likely follow-up**

"Maximal rectangle of 1s in a binary matrix" reduces to this: treat each row as the base of a histogram of consecutive 1s above it, and run this function per row.

Q274 TC-070 • Advanced

Find the maximum in every sliding window of size k.

⚡ **Quick answer**

Use a deque of indices with decreasing values: pop from the back while the new value is larger, pop from the front when the index leaves the window; the front is always the window maximum. O(n).

Java

```
static int[] maxSlidingWindow(int[] a, int k) {
    int[] out = new int[a.length - k + 1];
    Deque<Integer> dq = new ArrayDeque<>();
    for (int i = 0; i < a.length; i++) {
        if (!dq.isEmpty() && dq.peekFirst() <= i - k) dq.pollFirst();     // left the window
        while (!dq.isEmpty() && a[dq.peekLast()] <= a[i]) dq.pollLast();  // can never be max again
        dq.offerLast(i);
        if (i >= k - 1) out[i - k + 1] = a[dq.peekFirst()];
    }
    return out;
}
System.out.println(Arrays.toString(maxSlidingWindow(new int[]{1, 3, -1, -3, 5, 3, 6, 7}, 3)));
```

▶ **Output**

```
[3, 3, 5, 5, 6, 7]
```


© TechTribers. For personal use only. Do not redistribute.

Sliding window maximum (k = 3): deque keeps indices of decreasing values; front = max



`max per window: 3 3 5 5 6 7`

pop back while smaller than new · pop front when it leaves the window · O(n)

⚠️ **Interview trap**

A `PriorityQueue` also works but is O(n log k) and needs lazy deletion of expired elements. The monotonic deque is the expected O(n) answer.

Q275 TC-071

• Advanced

Merge K sorted linked lists.

⚡ **Quick answer**

Push the head of each list into a min-heap ordered by value; repeatedly poll the smallest node, append it, and push its successor. O(N log k) for N total nodes and k lists, O(k) heap space.

Java

```
static final class Node { int val; Node next; Node(int v, Node n) { val = v; next = n; } }
static Node of(int... v) { Node h = null; for (int i = v.length - 1; i >= 0; i--) h = new Node(v[i], h); return h; }

static Node mergeK(List<Node> lists) {
    PriorityQueue<Node> pq = new PriorityQueue<>(Comparator.comparingInt(n -> n.val));
    for (Node n : lists) if (n != null) pq.offer(n);
    Node dummy = new Node(0, null), tail = dummy;
    while (!pq.isEmpty()) {
        Node n = pq.poll();
        tail = tail.next = n;
        if (n.next != null) pq.offer(n.next);
    }
    return dummy.next;
}
StringBuilder sb = new StringBuilder();
for (Node n = mergeK(List.of(of(1, 4, 5), of(1, 3, 4), of(2, 6))); n != null; n = n.next)
sb.append(n.val).append(' ');
System.out.println(sb.toString().trim());
```

▶ **Output**

```
1 1 2 3 4 4 5 6
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> Divide and conquer (merge lists pairwise, like merge sort) has the same O(N log k) complexity without a heap. It's a good second answer if asked for alternatives.

Q276 TC-072 • Advanced

Find the median of a data stream.

> ⚡ **Quick answer**
> Keep two heaps: a max-heap for the lower half and a min-heap for the upper half, balanced so their sizes differ by at most one. The median is the top of the bigger heap or the average of both tops. Add O(log n), median O(1).

Java

```
static final class MedianFinder {
    private final PriorityQueue<Integer> low = new PriorityQueue<>(Comparator.reverseOrder());   //
max-heap
    private final PriorityQueue<Integer> high = new PriorityQueue<>();
// min-heap
    void add(int x) {
        low.offer(x);
        high.offer(low.poll());                        // keep every low <= every high
        if (high.size() > low.size()) low.offer(high.poll());
    }
    double median() {
        return low.size() > high.size() ? low.peek() : (low.peek() + (long) high.peek()) / 2.0;
    }
}
MedianFinder mf = new MedianFinder();
StringBuilder sb = new StringBuilder();
for (int x : new int[]{5, 15, 1, 3, 8}) { mf.add(x); sb.append(mf.median()).append(' '); }
System.out.println(sb.toString().trim());
```

▶ **Output**

```
5.0 10.0 5.0 4.0 5.0
```



© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

`(low.peek() + high.peek()) / 2` overflows for large ints and truncates the .5. Widen to `long` and divide by `2.0`.

Q277 TC-073 • Intermediate

Implement a min-heap from scratch.

⚡ **Quick answer**

Store a complete binary tree in an array: children of `i` are `2i + 1` and `2i + 2`, parent is `(i - 1) / 2`. `offer` appends and sifts up; `poll` moves the last element to the root and sifts down. Both O(log n); `peek` is O(1).

Java

```
static final class MinHeap {
    private int[] a = new int[8];
    private int size;
    void offer(int x) {
        if (size == a.length) a = Arrays.copyOf(a, size * 2);
        a[size] = x;
        for (int i = size++; i > 0 && a[(i - 1) / 2] > a[i]; i = (i - 1) / 2) swap(i, (i - 1) / 2);
    }
    int poll() {
        if (size == 0) throw new NoSuchElementException();
        int min = a[0];
        a[0] = a[--size];
        for (int i = 0; ; ) {
            int l = 2 * i + 1, r = l + 1, s = i;
            if (l < size && a[l] < a[s]) s = l;
            if (r < size && a[r] < a[s]) s = r;
            if (s == i) break;
            swap(i, s); i = s;
        }
        return min;
    }
    private void swap(int i, int j) { int t = a[i]; a[i] = a[j]; a[j] = t; }
}
MinHeap h = new MinHeap();
for (int x : new int[]{9, 4, 7, 1, 8, 2, 6, 3, 5}) h.offer(x);
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 9; i++) sb.append(h.poll()).append(' ');
System.out.println(sb.toString().trim());
```

▶ **Output**

```
1 2 3 4 5 6 7 8 9
```


© TechTribers. For personal use only. Do not redistribute.

🎯 **Likely follow-up**

Building a heap from n items with repeated `offer` is O(n log n); bottom-up heapify (sift down from the last parent) is O(n). Heap sort uses the latter.

Section 04

Linked Lists, Advanced

Combining the basic pointer tricks (fast/slow, reversal, dummy heads) into harder problems.

Questions 278–280 · 3 questions



Q278 TC-074

• Intermediate

Check if a linked list is a palindrome in O(1) extra space.

⚡ **Quick answer**

Find the middle with fast/slow pointers, reverse the second half, compare it with the first half, then reverse it back to restore the list. O(n) time, O(1) space.

Java

```
static final class Node { int val; Node next; Node(int v, Node n) { val = v; next = n; } }
static Node of(int... v) { Node h = null; for (int i = v.length - 1; i >= 0; i--) h = new Node(v[i], h); return h; }
static String str(Node h) { StringBuilder sb = new StringBuilder(); for (; h != null; h = h.next) sb.append(h.val).append(h.next != null ? "->" : ""); return sb.toString(); }
static Node reverse(Node h) { Node p = null; while (h != null) { Node n = h.next; h.next = p; p = h; h = n; } return p; }

static boolean isPalindrome(Node head) {
    Node slow = head, fast = head;
    while (fast != null && fast.next != null) { slow = slow.next; fast = fast.next.next; }
    Node second = reverse(slow), p1 = head, p2 = second;
    boolean ok = true;
    while (p2 != null) { if (p1.val != p2.val) { ok = false; break; } p1 = p1.next; p2 = p2.next; }
    reverse(second);                                    // restore the input
    return ok;
}
Node l = of(1, 2, 3, 2, 1);
System.out.println(isPalindrome(l) + " " + isPalindrome(of(1, 2, 2, 1)) + " " + isPalindrome(of(1, 2, 3)) + " | list intact: " + str(l));
```

▶ **Output**

```
true true false | list intact: 1->2->3->2->1
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> Restoring the list afterwards is what senior interviewers look for: a check method shouldn't mutate its input.

➕ Q279 TC-075 • Intermediate

Add two numbers stored as reversed-digit linked lists.

> ⚡ **Quick answer**
> Walk both lists together, adding digits plus the carry; append `sum % 10` and carry `sum / 10`. Continue while either list or the carry remains. O(max(m, n)).

Java

```
static final class Node { int val; Node next; Node(int v, Node n) { val = v; next = n; } }
static Node of(int... v) { Node h = null; for (int i = v.length - 1; i >= 0; i--) h = new Node(v[i], h); return h; }
static String str(Node h) { StringBuilder sb = new StringBuilder(); for (; h != null; h = h.next) sb.append(h.val).append(h.next != null ? "->" : ""); return sb.toString(); }
static Node add(Node a, Node b) {
    Node dummy = new Node(0, null), tail = dummy;
    int carry = 0;
    while (a != null || b != null || carry != 0) {
        int sum = carry + (a != null ? a.val : 0) + (b != null ? b.val : 0);
        tail = tail.next = new Node(sum % 10, null);
        carry = sum / 10;
        if (a != null) a = a.next;
        if (b != null) b = b.next;
    }
    return dummy.next;
}
System.out.println(str(add(of(2, 4, 3), of(5, 6, 4))) + "   (342 + 465 = 807)");
System.out.println(str(add(of(9, 9, 9, 9), of(9, 9))) + "   (9999 + 99 = 10098)");
```

▶ **Output**

```
7->0->8   (342 + 465 = 807)
8->9->0->0->1   (9999 + 99 = 10098)
```

> ⚠️ **Interview trap**
> Converting the lists to `long` and adding fails for numbers with more than 18 digits. Digit-by-digit addition works for any length.

© TechTribers. For personal use only. Do not redistribute.

Q280 TC-076

• Advanced

Sort a linked list in O(n log n) time.

⚡ Quick answer

Merge sort: split the list at the middle (fast/slow pointers), sort both halves recursively, and merge them. O(n log n) time; O(log n) recursion stack, and no array copying, which is why merge sort suits linked lists better than quicksort.

Java

```
static final class Node { int val; Node next; Node(int v, Node n) { val = v; next = n; } }
static Node of(int... v) { Node h = null; for (int i = v.length - 1; i >= 0; i--) h = new Node(v[i],
h); return h; }
static String str(Node h) { StringBuilder sb = new StringBuilder(); for (; h != null; h = h.next)
sb.append(h.val).append(h.next != null ? "->" : ""); return sb.toString(); }
static Node merge(Node a, Node b) {
    Node d = new Node(0, null), t = d;
    while (a != null && b != null) { if (a.val <= b.val) { t.next = a; a = a.next; } else { t.next =
b; b = b.next; } t = t.next; }
    t.next = a != null ? a : b;
    return d.next;
}
static Node sort(Node head) {
    if (head == null || head.next == null) return head;
    Node slow = head, fast = head.next;                // fast starts one ahead: left half <=
right half
    while (fast != null && fast.next != null) { slow = slow.next; fast = fast.next.next; }
    Node right = slow.next;
    slow.next = null;
    return merge(sort(head), sort(right));
}
System.out.println(str(sort(of(4, 2, 1, 3))) + " | " + str(sort(of(-1, 5, 3, 4, 0))));
```

▶ Output

```
1->2->3->4 | -1->0->3->4->5
```

⚠️ Interview trap

Starting `fast` at `head` (not `head.next`) never splits a two-node list, causing infinite recursion and `StackOverflowError`.

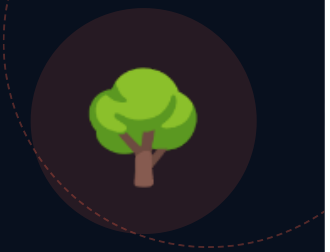
© TechTribers. For personal use only. Do not redistribute.

Section 05

Trees & Tries

Recursive tree problems that test whether you can return one thing while tracking another, plus tree construction and serialization.

Questions 281–287 · 7 questions



Q281 TC-077

• Intermediate

Find the diameter of a binary tree.

⚡ Quick answer

The diameter (longest path between any two nodes, in edges) passes through some node as `leftHeight + rightHeight`. Compute heights recursively and update a global maximum at each node. O(n).

Java

```
record TreeNode(int val, TreeNode left, TreeNode right) {
    TreeNode(int val) { this(val, null, null); }
}
static int best;
static int height(TreeNode n) {
    if (n == null) return 0;
    int l = height(n.left()), r = height(n.right());
    best = Math.max(best, l + r);                    // path through this node
    return 1 + Math.max(l, r);
}
static int diameter(TreeNode root) { best = 0; height(root); return best; }

TreeNode root = new TreeNode(1, new TreeNode(2, new TreeNode(4), new TreeNode(5)), new TreeNode(3));
System.out.println(diameter(root));
```

▶ Output

```
3
```

⚠️ Interview trap

The longest path doesn't have to pass through the root. Returning `height(left) + height(right)` of the root only is the common wrong answer.

© TechTribers. For personal use only. Do not redistribute.

💰 Q282 TC-078 • Advanced

Find the maximum path sum in a binary tree.

⚡ Quick answer

At each node, the best path through it is `val + max(0, leftGain) + max(0, rightGain)`; update the global best with that, but return `val + max(0, max(leftGain, rightGain))` upward, because a path can only continue along one branch. O(n).

Java

```
record TreeNode(int val, TreeNode left, TreeNode right) {
    TreeNode(int val) { this(val, null, null); }
}
static int best;
static int gain(TreeNode n) {
    if (n == null) return 0;
    int l = Math.max(0, gain(n.left())), r = Math.max(0, gain(n.right()));   // drop negative
branches
    best = Math.max(best, n.val() + l + r);
    return n.val() + Math.max(l, r);
}
static int maxPathSum(TreeNode root) { best = Integer.MIN_VALUE; gain(root); return best; }

TreeNode t = new TreeNode(-10, new TreeNode(9), new TreeNode(20, new TreeNode(15), new
TreeNode(7)));
System.out.println(maxPathSum(t) + " " + maxPathSum(new TreeNode(-3)));
```

▶ Output

```
42 -3
```

⚠️ Interview trap

Initialising `best = 0` returns 0 for a tree with only negative values. Start from `Integer.MIN_VALUE`.

💾 Q283 TC-079 • Advanced

Serialize and deserialize a binary tree.

⚡ Quick answer

Preorder traversal writing a marker (e.g. `#`) for null children produces a string that uniquely encodes the tree. Deserialize by reading tokens in the same preorder from an iterator. O(n) both ways.

Java

```
record TreeNode(int val, TreeNode left, TreeNode right) {
    TreeNode(int val) { this(val, null, null); }
}
```


© TechTribers. For personal use only. Do not redistribute.

```
static String serialize(TreeNode n) {
    StringJoiner sj = new StringJoiner(",");
    write(n, sj);
    return sj.toString();
}
static void write(TreeNode n, StringJoiner sj) {
    if (n == null) { sj.add("#"); return; }
    sj.add(String.valueOf(n.val())); write(n.left(), sj); write(n.right(), sj);
}
static TreeNode deserialize(String s) { return read(Arrays.asList(s.split(",")).iterator()); }
static TreeNode read(Iterator<String> it) {
    String tok = it.next();
    if (tok.equals("#")) return null;
    int v = Integer.parseInt(tok);
    TreeNode l = read(it);
    TreeNode r = read(it);
    return new TreeNode(v, l, r);
}
TreeNode t = new TreeNode(1, new TreeNode(2), new TreeNode(3, new TreeNode(4), new TreeNode(5)));
String s = serialize(t);
System.out.println(s + " | round-trip equal: " + t.equals(deserialize(s)));
```

▶ Output

```
1,2,#,#,3,4,#,#,5,#,# | round-trip equal: true
```

💡 Pro tip

Because `TreeNode` is a record, `equals` compares the whole structure recursively, which makes the round-trip check a one-liner.

Q284 TC-080 • Intermediate

Find the k-th smallest element in a BST.

⚡ Quick answer

An in-order traversal of a BST visits values in ascending order, so stop at the k-th visited node. Iteratively with a stack this is O(h + k) time and O(h) space.

Java

```
record TreeNode(int val, TreeNode left, TreeNode right) {
    TreeNode(int val) { this(val, null, null); }
}
static int kthSmallest(TreeNode root, int k) {
    Deque<TreeNode> stack = new ArrayDeque<>();
    TreeNode cur = root;
    while (cur != null || !stack.isEmpty()) {
        while (cur != null) { stack.push(cur); cur = cur.left(); }
        cur = stack.pop();
        if (--k == 0) return cur.val();
        cur = cur.right();
```


© TechTribers. For personal use only. Do not redistribute.

```
    }
    throw new IllegalArgumentException("k > size");
}
//        5
//       / \
//      3   6
//     / \
//    2   4
//   /
//  1
TreeNode root = new TreeNode(5, new TreeNode(3, new TreeNode(2, new TreeNode(1), null), new
TreeNode(4)), new TreeNode(6));
System.out.println(kthSmallest(root, 1) + " " + kthSmallest(root, 3) + " " + kthSmallest(root, 6));
```

▶ Output

```
1 3 6
```

🎯 Likely follow-up

If the tree is modified often and k-th queries are frequent, augment each node with its subtree size: then each query is O(h) without traversal.

🏗 Q285 TC-081 • Advanced

Construct a binary tree from preorder and inorder traversals.

⚡ Quick answer

The first preorder value is the root; its position in the inorder array splits the left and right subtrees. Recurse with index ranges and a value→index map for O(1) lookups. O(n) time.

Java

```
record TreeNode(int val, TreeNode left, TreeNode right) {
    TreeNode(int val) { this(val, null, null); }
}
static int pre;
static TreeNode build(int[] preorder, int[] inorder) {
    Map<Integer, Integer> idx = new HashMap<>();
    for (int i = 0; i < inorder.length; i++) idx.put(inorder[i], i);
    pre = 0;
    return build(preorder, idx, 0, inorder.length - 1);
}
static TreeNode build(int[] preorder, Map<Integer, Integer> idx, int lo, int hi) {
    if (lo > hi) return null;
    int val = preorder[pre++];
    int mid = idx.get(val);
    TreeNode left = build(preorder, idx, lo, mid - 1);       // left subtree is consumed first
    TreeNode right = build(preorder, idx, mid + 1, hi);
    return new TreeNode(val, left, right);
}
```


© TechTribers. For personal use only. Do not redistribute.

```
TreeNode t = build(new int[]{3, 9, 20, 15, 7}, new int[]{9, 3, 15, 20, 7});
System.out.println(t);
```

▶ Output

```
TreeNode[val=3, left=TreeNode[val=9, left=null, right=null], right=TreeNode[val=20,
left=TreeNode[val=15, left=null, right=null], right=TreeNode[val=7, left=null, right=null]]]
```

⚠️ Interview trap

This only works when values are unique. With duplicates, the inorder position of the root is ambiguous and the tree can't be reconstructed uniquely.

Q286 TC-082 • Basic

Invert (mirror) a binary tree.

⚡ Quick answer

Swap the left and right children of every node, recursively or with a BFS queue. O(n) time, O(h) space. It became famous after a well-known engineer tweeted he failed it in an interview.

Java

```
record TreeNode(int val, TreeNode left, TreeNode right) {
    TreeNode(int val) { this(val, null, null); }
}
static TreeNode invert(TreeNode n) {
    return n == null ? null : new TreeNode(n.val(), invert(n.right()), invert(n.left()));
}
static List<Integer> levelOrder(TreeNode root) {
    List<Integer> out = new ArrayList<>();
    Deque<TreeNode> q = new ArrayDeque<>(List.of(root));
    while (!q.isEmpty()) {
        TreeNode n = q.poll(); out.add(n.val());
        if (n.left() != null) q.offer(n.left());
        if (n.right() != null) q.offer(n.right());
    }
    return out;
}
TreeNode t = new TreeNode(4, new TreeNode(2, new TreeNode(1), new TreeNode(3)), new TreeNode(7, new
TreeNode(6), new TreeNode(9)));
System.out.println(levelOrder(t) + " -> " + levelOrder(invert(t)));
```

▶ Output

```
[4, 2, 7, 1, 3, 6, 9] -> [4, 7, 2, 9, 6, 3, 1]
```


© TechTribers. For personal use only. Do not redistribute.

🎯 **Likely follow-up**

"Is the tree symmetric?" is the follow-up: compare `left` with the mirror of `right` recursively (`a.left` vs `b.right` and `a.right` vs `b.left`).

Q287 TC-083 • Intermediate

Implement a Trie (prefix tree) with insert, search and startsWith.

⚡ **Quick answer**

Each node holds children (array of 26 or a map) and an end-of-word flag. Insert and search walk one node per character: O(L) for a word of length L, independent of how many words are stored.

Java

```
static final class Trie {
    private static final class Node { final Node[] next = new Node[26]; boolean word; }
    private final Node root = new Node();

    void insert(String w) {
        Node n = root;
        for (char c : w.toCharArray()) n = n.next[c - 'a'] == null ? (n.next[c - 'a'] = new Node())
: n.next[c - 'a'];
        n.word = true;
    }
    private Node walk(String s) {
        Node n = root;
        for (char c : s.toCharArray()) if ((n = n.next[c - 'a']) == null) return null;
        return n;
    }
    boolean search(String w) { Node n = walk(w); return n != null && n.word; }
    boolean startsWith(String p) { return walk(p) != null; }
    List<String> suggest(String prefix) {                    // autocomplete
        List<String> out = new ArrayList<>();
        collect(walk(prefix), new StringBuilder(prefix), out);
        return out;
    }
    private void collect(Node n, StringBuilder sb, List<String> out) {
        if (n == null) return;
        if (n.word) out.add(sb.toString());
        for (int i = 0; i < 26; i++) if (n.next[i] != null) { sb.append((char) ('a' + i));
collect(n.next[i], sb, out); sb.deleteCharAt(sb.length() - 1); }
    }
}
Trie t = new Trie();
for (String w : List.of("java", "javascript", "jvm", "kafka", "jar")) t.insert(w);
System.out.println(t.search("java") + " " + t.search("jav") + " " + t.startsWith("jav") + " " +
t.suggest("ja"));
```

▶ **Output**

```
true false true [jar, java, javascript]
```


© TechTribers. For personal use only. Do not redistribute.

💡 **Pro tip**

Real uses: autocomplete, spell checkers, IP routing (longest prefix match), and the dictionary step of "word search II".

Section 06

Graphs

BFS, topological sort, shortest paths and union-find: the four graph tools that cover most interview graph problems.

Questions 288–293 · 6 questions



Q288 TC-084 • Intermediate

Rotting oranges: minimum minutes until all oranges rot (multi-source BFS).

⚡ **Quick answer**

Put all initially rotten cells in the queue at once and run BFS level by level; each level is one minute. Count fresh oranges and return −1 if any remain unreachable. O(rows × cols).

Java

```java
static int orangesRotting(int[][] g) {
    Deque<int[]> q = new ArrayDeque<>();
    int fresh = 0;
    for (int r = 0; r < g.length; r++)
        for (int c = 0; c < g[0].length; c++) {
            if (g[r][c] == 2) q.offer(new int[]{r, c});
            else if (g[r][c] == 1) fresh++;
        }
    int minutes = 0;
    int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
    while (!q.isEmpty() && fresh > 0) {
        for (int i = q.size(); i > 0; i--) {
            int[] cell = q.poll();
            for (int[] d : dirs) {
                int r = cell[0] + d[0], c = cell[1] + d[1];
                if (r < 0 || c < 0 || r >= g.length || c >= g[0].length || g[r][c] != 1) continue;
                g[r][c] = 2; fresh--; q.offer(new int[]{r, c});
            }
        }
        minutes++;
    }
    return fresh == 0 ? minutes : -1;
}
```


© TechTribers. For personal use only. Do not redistribute.

```
System.out.println(orangesRotting(new int[][]{{2, 1, 1}, {1, 1, 0}, {0, 1, 1}}) + " " +
orangesRotting(new int[][]{{2, 1, 1}, {0, 1, 1}, {1, 0, 1}}));
```

▶ Output

```
4 -1
```

💡 Pro tip

BFS gives shortest paths in **unweighted** graphs. Multi-source BFS (all sources start in the queue together) answers "distance to the nearest X" for every cell in one pass.

Q289 TC-085 • Advanced

Word ladder: shortest transformation sequence from one word to another.

⚡ Quick answer

BFS over words where an edge means "differs by one letter". Generate neighbours by changing each position to 'a'–'z' and checking a `HashSet` of the dictionary; remove words when visited. O(N × L × 26).

Java

```
static int ladderLength(String begin, String end, List<String> words) {
    Set<String> dict = new HashSet<>(words);
    if (!dict.contains(end)) return 0;
    Deque<String> q = new ArrayDeque<>(List.of(begin));
    for (int steps = 1; !q.isEmpty(); steps++) {
        for (int i = q.size(); i > 0; i--) {
            char[] w = q.poll().toCharArray();
            for (int p = 0; p < w.length; p++) {
                char orig = w[p];
                for (char c = 'a'; c <= 'z'; c++) {
                    w[p] = c;
                    String next = new String(w);
                    if (next.equals(end)) return steps + 1;
                    if (dict.remove(next)) q.offer(next);        // remove = mark visited
                }
                w[p] = orig;
            }
        }
    }
    return 0;
}
System.out.println(ladderLength("hit", "cog", List.of("hot", "dot", "dog", "lot", "log", "cog")) + "
"
        + ladderLength("hit", "cog", List.of("hot", "dot", "dog", "lot", "log")));
```

▶ Output

```
5 0
```


© TechTribers. For personal use only. Do not redistribute.

Q290 TC-086

• Intermediate

Course schedule: topological sort and cycle detection in a directed graph.

⚡ Quick answer

Kahn's algorithm: compute in-degrees, start a queue with all zero-in-degree nodes, and repeatedly remove a node and decrement its neighbours' in-degrees. If not all nodes are output, the graph has a cycle. O(V + E).

Real-world topological sorts: Maven/Gradle build order, Spring bean initialisation order, task schedulers, and spreadsheet formula evaluation.

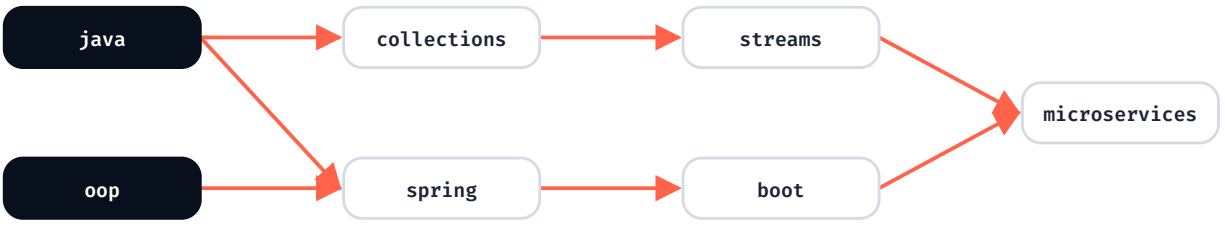
Java

```
static List<Integer> topoOrder(int n, int[][] prereqs) {        // prereq {a, b}: b before a
    List<List<Integer>> adj = new ArrayList<>();
    for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
    int[] indeg = new int[n];
    for (int[] p : prereqs) { adj.get(p[1]).add(p[0]); indeg[p[0]]++; }
    Deque<Integer> q = new ArrayDeque<>();
    for (int i = 0; i < n; i++) if (indeg[i] == 0) q.offer(i);
    List<Integer> order = new ArrayList<>();
    while (!q.isEmpty()) {
        int u = q.poll();
        order.add(u);
        for (int v : adj.get(u)) if (--indeg[v] == 0) q.offer(v);
    }
    return order.size() == n ? order : List.of();                // empty = cycle
}
System.out.println(topoOrder(4, new int[][]{{1, 0}, {2, 0}, {3, 1}, {3, 2}}) + " " + topoOrder(2,
new int[][]{{1, 0}, {0, 1}}));
```

▶ Output

```
[0, 1, 2, 3] []
```

Topological sort (Kahn): repeatedly take nodes with in-degree 0



order: java, oop, collections, spring, streams, boot, microservices · cycle ⇔ not all nodes output

© TechTribers. For personal use only. Do not redistribute.

Q291 TC-087 • Advanced

Find shortest paths with Dijkstra's algorithm.

⚡ Quick answer

Use a min-heap of (distance, node); pop the closest node, skip stale entries, and relax its edges. Works for non-negative weights in O((V + E) log V). For negative weights use Bellman–Ford.

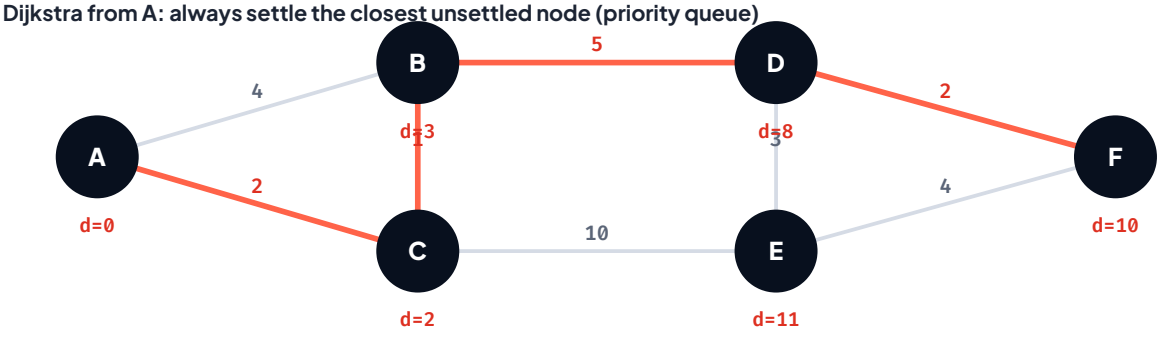
Java

```
record Edge(int to, int w) { }

static int[] dijkstra(List<List<Edge>> g, int src) {
    int[] dist = new int[g.size()];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;
    PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a -> a[1]));
    pq.offer(new int[]{src, 0});
    while (!pq.isEmpty()) {
        int[] cur = pq.poll();
        int u = cur[0];
        if (cur[1] > dist[u]) continue;                    // stale entry
        for (Edge e : g.get(u))
            if (dist[u] + e.w() < dist[e.to()]) { dist[e.to()] = dist[u] + e.w(); pq.offer(new int[]
{e.to(), dist[e.to()]}); }
    }
    return dist;
}
// A=0 B=1 C=2 D=3 E=4 F=5
List<List<Edge>> g = new ArrayList<>();
for (int i = 0; i < 6; i++) g.add(new ArrayList<>());
int[][] edges = {{0, 1, 4}, {0, 2, 2}, {2, 1, 1}, {1, 3, 5}, {2, 4, 10}, {3, 4, 3}, {3, 5, 2}, {4,
5, 4}};
for (int[] e : edges) { g.get(e[0]).add(new Edge(e[1], e[2])); g.get(e[1]).add(new Edge(e[0],
e[2])); }
System.out.println(Arrays.toString(dijkstra(g, 0)));
```

▶ Output

```
[0, 3, 2, 8, 11, 10]
```



A → C → B → D → F = 10 · O((V + E) log V) · fails with negative edges (use Bellman-Ford)

© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

Java's `PriorityQueue` has no decrease-key operation. Pushing a new entry and skipping stale ones on poll (as above) is the standard workaround.

🤝 **Q292** TC-088 • Advanced

Implement Union-Find and count connected components.

⚡ **Quick answer**

Each element points to a parent; `find` follows parents to the root with path compression, and `union` links two roots by rank. With both optimisations, operations are nearly O(1) (inverse Ackermann).

Java

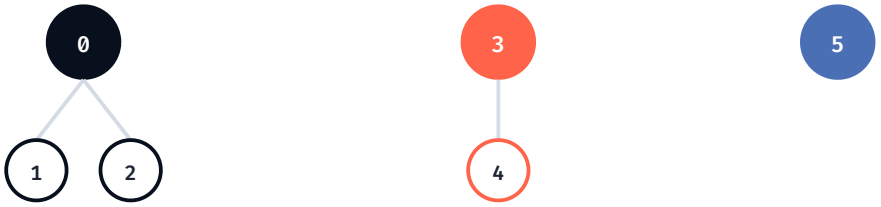
```
static final class DSU {
    private final int[] parent, rank;
    int components;
    DSU(int n) { parent = new int[n]; rank = new int[n]; components = n; for (int i = 0; i < n; i++)
parent[i] = i; }
    int find(int x) { return parent[x] == x ? x : (parent[x] = find(parent[x])); }   // path
compression
    boolean union(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return false;                    // already connected: this edge forms a cycle
        if (rank[ra] < rank[rb]) { int t = ra; ra = rb; rb = t; }
        parent[rb] = ra;
        if (rank[ra] == rank[rb]) rank[ra]++;
        components--;
        return true;
    }
}
DSU d = new DSU(6);
d.union(0, 1); d.union(1, 2); d.union(3, 4);
System.out.println("components=" + d.components + " cycle on (0,2)? " + !d.union(0, 2) + " after
union(2,4): " + (d.union(2, 4) ? d.components : -1));
```

▶ **Output**

```
components=3 cycle on (0,2)? true after union(2,4): 2
```


© TechTribers. For personal use only. Do not redistribute.

Union-Find: each set is a tree; find() walks to the root (path compression), union() links roots by rank



3 components → union(2, 4) merges the first two → 2 components · ~O(α(n)) per operation

💡 **Pro tip**

Use Union-Find for dynamic connectivity: Kruskal's minimum spanning tree, detecting cycles in undirected graphs, grouping accounts by shared emails, and "number of provinces".

🎨 **Q293** TC-089 • Intermediate

Check whether a graph is bipartite (two-colourable).

⚡ **Quick answer**

BFS (or DFS) from every uncoloured node, assigning alternating colours; if an edge connects two nodes of the same colour, the graph isn't bipartite. O(V + E). A graph is bipartite exactly when it has no odd-length cycle.

Java

```
static boolean isBipartite(int[][] graph) {
    int[] color = new int[graph.length];                    // 0 = uncoloured, 1 / -1
    for (int s = 0; s < graph.length; s++) {
        if (color[s] != 0) continue;
        color[s] = 1;
        Deque<Integer> q = new ArrayDeque<>(List.of(s));
        while (!q.isEmpty()) {
            int u = q.poll();
            for (int v : graph[u]) {
                if (color[v] == color[u]) return false;
                if (color[v] == 0) { color[v] = -color[u]; q.offer(v); }
            }
        }
    }
    return true;
}
System.out.println(isBipartite(new int[][]{{1, 3}, {0, 2}, {1, 3}, {0, 2}}) + " " + isBipartite(new
int[][]{{1, 2, 3}, {0, 2}, {0, 1, 3}, {0, 2}}));
```

▶ **Output**

```
true false
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

Starting BFS only from node 0 misses disconnected components. Loop over every node.

Section 07

Dynamic Programming

Define the state, write the recurrence, then optimise space: the eight DP problems that come up most.

Questions 294–301 · 8 questions



Q294 TC-090 • Intermediate

House robber: maximum sum of non-adjacent elements.

⚡ **Quick answer**

`dp[i] = max(dp[i-1], dp[i-2] + nums[i])` : either skip house i or rob it plus the best up to i−2. Only the last two values are needed, so O(n) time, O(1) space.

Java

```
static int rob(int[] nums) {
    int prev2 = 0, prev1 = 0;                       // best up to i-2 and i-1
    for (int x : nums) {
        int cur = Math.max(prev1, prev2 + x);
        prev2 = prev1;
        prev1 = cur;
    }
    return prev1;
}
static int climbStairs(int n) {                     // same 1D-DP shape: ways(n) = ways(n-1) +
ways(n-2)
    int a = 1, b = 1;
    for (int i = 2; i <= n; i++) { int c = a + b; a = b; b = c; }
    return b;
}
System.out.println(rob(new int[]{2, 7, 9, 3, 1}) + " " + rob(new int[]{1, 2, 3, 1}) + " " +
climbStairs(5) + " " + climbStairs(10));
```

▶ **Output**

```
12 4 8 89
```


© TechTribers. For personal use only. Do not redistribute.

Q295 TC-091

• Advanced

Longest increasing subsequence in O(n log n).

⚡ Quick answer

Maintain `tails`, where `tails[i]` is the smallest possible tail of an increasing subsequence of length i+1. For each number, binary-search its position in `tails` and replace (or append). The answer is `tails.size()`.

Java

```
static int lis(int[] nums) {
    int[] tails = new int[nums.length];
    int size = 0;
    for (int x : nums) {
        int i = Arrays.binarySearch(tails, 0, size, x);
        if (i < 0) i = -(i + 1);                          // insertion point
        tails[i] = x;
        if (i == size) size++;
    }
    return size;
}
static int lisQuadratic(int[] nums) {                     // O(n^2) DP for comparison
    int[] dp = new int[nums.length];
    int best = 0;
    for (int i = 0; i < nums.length; i++) {
        dp[i] = 1;
        for (int j = 0; j < i; j++) if (nums[j] < nums[i]) dp[i] = Math.max(dp[i], dp[j] + 1);
        best = Math.max(best, dp[i]);
    }
    return best;
}
int[] a = {10, 9, 2, 5, 3, 7, 101, 18};
System.out.println(lis(a) + " " + lisQuadratic(a) + " " + lis(new int[]{7, 7, 7, 7}));
```

▶ Output

```
4 4 1
```

⚠️ Interview trap

`tails` is not itself a valid subsequence; it only gives the length. To print the actual subsequence, also store predecessor indices.

© TechTribers. For personal use only. Do not redistribute.

Q296 TC-092

• Advanced

0/1 knapsack: maximise value within a weight capacity.

⚡ Quick answer

`dp[c]` = best value with capacity c. For each item, iterate capacities **downwards** so each item is used at most once: `dp[c] = max(dp[c], dp[c - w] + v)`. O(n × W) time, O(W) space.

Java

```
static int knapsack(int[] w, int[] v, int cap) {
    int[] dp = new int[cap + 1];
    for (int i = 0; i < w.length; i++)
        for (int c = cap; c >= w[i]; c--)                    // downwards: 0/1, not unbounded
            dp[c] = Math.max(dp[c], dp[c - w[i]] + v[i]);
    return dp[cap];
}
static boolean canPartition(int[] nums) {                    // equal-subset-sum: knapsack on
booleans
    int sum = Arrays.stream(nums).sum();
    if (sum % 2 != 0) return false;
    boolean[] dp = new boolean[sum / 2 + 1];
    dp[0] = true;
    for (int x : nums) for (int c = sum / 2; c >= x; c--) dp[c] |= dp[c - x];
    return dp[sum / 2];
}
System.out.println(knapsack(new int[]{1, 3, 4}, new int[]{15, 20, 30}, 4) + " " + knapsack(new int[]
{1, 2, 3}, new int[]{10, 15, 40}, 6));
System.out.println(canPartition(new int[]{1, 5, 11, 5}) + " " + canPartition(new int[]{1, 2, 3,
5}));
```

▶ Output

```
35 65
true false
```

0/1 knapsack (cap 4): dp[i][c] = max(skip item i, take it: dp[i−1][c − w] + v)

	c=0	c=1	c=2	c=3	c=4
none	0	0	0	0	0
w1 v15	0	15	15	15	15
w3 v20	0	15	15	20	35
w4 v30	0	15	15	20	**35**

best = 35

items w1 + w3 (15 + 20)

O(n × capacity)

⚠️ Interview trap

Looping capacity **upwards** in the 1D version lets an item be reused, which silently solves the unbounded knapsack instead.

© TechTribers. For personal use only. Do not redistribute.

✏️ Q297 TC-093 • Advanced

Edit distance (Levenshtein) between two strings.

⚡ Quick answer

`dp[i][j]` = edits to turn the first i characters of A into the first j of B. Same character: `dp[i-1][j-1]`; otherwise 1 + min(delete, insert, replace). O(m × n) time; O(n) space with a rolling row.

Used in spell correction ("did you mean…"), fuzzy search, DNA alignment, and Elasticsearch's fuzzy queries (which use a Levenshtein automaton).

Java

```
static int editDistance(String a, String b) {
    int[] prev = new int[b.length() + 1], cur = new int[b.length() + 1];
    for (int j = 0; j <= b.length(); j++) prev[j] = j;
    for (int i = 1; i <= a.length(); i++) {
        cur[0] = i;
        for (int j = 1; j <= b.length(); j++)
            cur[j] = a.charAt(i - 1) == b.charAt(j - 1) ? prev[j - 1] : 1 + Math.min(prev[j - 1],
Math.min(prev[j], cur[j - 1]));
        int[] t = prev; prev = cur; cur = t;
    }
    return prev[b.length()];
}
System.out.println(editDistance("horse", "ros") + " " + editDistance("intention", "execution") + " "
+ editDistance("", "abc") + " " + editDistance("kafka", "kafka"));
```

▶ Output

```
3 5 3 0
```

Edit distance("horse", "ros") = 3: dp[i][j] = same ? diag : 1 + min(delete, insert, replace)

	∅	r	o	s
∅	0	1	2	3
h	1	1	2	3
o	2	2	1	2
r	3	2	2	2
s	4	3	3	2
e	5	4	4	3

horse → rorse (replace h→r)
rorse → rose (delete r)
rose → ros (delete e)

O(m × n) time and space

© TechTribers. For personal use only. Do not redistribute.

Q298 TC-094

• Intermediate

Word break: can a string be segmented into dictionary words?

⚡ Quick answer

`dp[i]` is true if the prefix of length i can be segmented: `dp[i] = OR over j < i of dp[j] && dict.contains(s[j..i))` . Limit j to the maximum word length. O(n × maxLen) lookups.

Java

```
static boolean wordBreak(String s, Set<String> dict) {
    int maxLen = dict.stream().mapToInt(String::length).max().orElse(0);
    boolean[] dp = new boolean[s.length() + 1];
    dp[0] = true;
    for (int i = 1; i <= s.length(); i++)
        for (int j = Math.max(0, i - maxLen); j < i && !dp[i]; j++)
            dp[i] = dp[j] && dict.contains(s.substring(j, i));
    return dp[s.length()];
}
System.out.println(wordBreak("techtribers", Set.of("tech", "tribe", "tribers", "rs")) + " "
        + wordBreak("applepenapple", Set.of("apple", "pen")) + " "
        + wordBreak("catsandog", Set.of("cats", "dog", "sand", "and", "cat")));
```

▶ Output

```
true true false
```

⚠️ Interview trap

Plain recursion without memoization is exponential on inputs like "aaaaaaaaab" with dictionary {a, aa, aaa}. Always cache results per start index.

Q299 TC-095

• Intermediate

Count unique paths in a grid (with and without obstacles).

⚡ Quick answer

Paths to a cell = paths from above + paths from the left; obstacle cells have 0. A single row array suffices: `dp[j] += dp[j - 1]` . O(m × n) time, O(n) space. Without obstacles it's also C(m+n−2, m−1).

Java

```
static long uniquePaths(int[][] grid) {                // 1 = obstacle
    int n = grid[0].length;
    long[] dp = new long[n];
    dp[0] = grid[0][0] == 0 ? 1 : 0;
    for (int[] row : grid)
        for (int j = 0; j < n; j++) {
            if (row[j] == 1) dp[j] = 0;
```


© TechTribers. For personal use only. Do not redistribute.

```
            else if (j > 0) dp[j] += dp[j - 1];
        }
    return dp[n - 1];
}
System.out.println(uniquePaths(new int[3][7]) + " " + uniquePaths(new int[][]{{0, 0, 0}, {0, 1, 0},
{0, 0, 0}}));
```

▶ Output

```
28 2
```

Q300 TC-096 • Intermediate

Decode ways: count decodings of a digit string (A=1 … Z=26).

⚡ Quick answer

`dp[i] = (s[i-1] != '0' ? dp[i-1] : 0) + (s[i-2..i] in 10..26 ? dp[i-2] : 0)`. Only two previous values are needed: O(n) time, O(1) space. Zeros are the whole difficulty.

Java

```
static int numDecodings(String s) {
    if (s.isEmpty() || s.charAt(0) == '0') return 0;
    int prev2 = 1, prev1 = 1;
    for (int i = 2; i <= s.length(); i++) {
        int cur = 0;
        if (s.charAt(i - 1) != '0') cur += prev1;
        int two = Integer.parseInt(s.substring(i - 2, i));
        if (two >= 10 && two <= 26) cur += prev2;
        prev2 = prev1;
        prev1 = cur;
    }
    return prev1;
}
for (String s : List.of("12", "226", "06", "10", "2101", "11106"))
    System.out.print(numDecodings(s) + " ");
```

▶ Output

```
2 3 0 1 1 2
```

Q301 TC-097 • Advanced

Maximum product subarray.

⚡ Quick answer

Track both the maximum and minimum product ending at each index, because multiplying by a negative number swaps them. Update `best` from the running maximum. O(n) time, O(1) space.

© TechTribers. For personal use only. Do not redistribute.

```java
static int maxProduct(int[] a) {
    int max = a[0], min = a[0], best = a[0];
    for (int i = 1; i < a.length; i++) {
        int x = a[i];
        if (x < 0) { int t = max; max = min; min = t; }        // negative flips the extremes
        max = Math.max(x, max * x);
        min = Math.min(x, min * x);
        best = Math.max(best, max);
    }
    return best;
}
System.out.println(maxProduct(new int[]{2, 3, -2, 4}) + " " + maxProduct(new int[]{-2, 0, -1}) + " "
+ maxProduct(new int[]{-2, 3, -4}));
```

▶ Output

```
6 0 24
```

⚠️ **Interview trap**

Applying Kadane directly (tracking only the maximum) fails on `[-2, 3, -4]`, where two negatives make the best product 24.

Section 08

Backtracking

Choose, explore, un-choose: one template for subsets, combinations and constraint puzzles.

Questions 302–304 · 3 questions



Q302 TC-098

• Intermediate

Generate all subsets (power set) of a set.

⚡ **Quick answer**

Backtracking: at each index, record the current subset, then for each later element include it, recurse, and remove it. 2^n subsets, so $O(n \cdot 2^n)$. Bitmasks (0 … 2^n-1) give the same result iteratively.

```java
static List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> out = new ArrayList<>();
    backtrack(nums, 0, new ArrayList<>(), out);
```

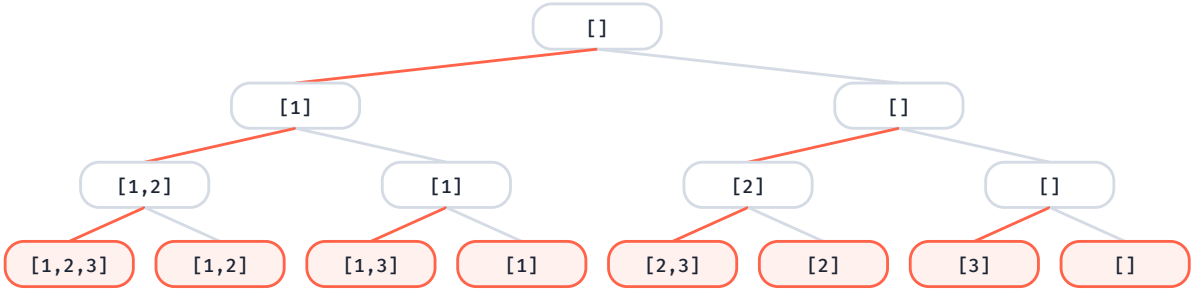

© TechTribers. For personal use only. Do not redistribute.

```
    return out;
}
static void backtrack(int[] nums, int start, List<Integer> cur, List<List<Integer>> out) {
    out.add(new ArrayList<>(cur));                    // copy: cur keeps changing
    for (int i = start; i < nums.length; i++) {
        cur.add(nums[i]);
        backtrack(nums, i + 1, cur, out);
        cur.remove(cur.size() - 1);
    }
}
static List<List<Integer>> subsetsBitmask(int[] nums) {
    List<List<Integer>> out = new ArrayList<>();
    for (int mask = 0; mask < 1 << nums.length; mask++) {
        List<Integer> s = new ArrayList<>();
        for (int i = 0; i < nums.length; i++) if ((mask >> i & 1) == 1) s.add(nums[i]);
        out.add(s);
    }
    return out;
}
System.out.println(subsets(new int[]{1, 2, 3}));
System.out.println(subsetsBitmask(new int[]{1, 2, 3}).size());
```

▶ Output

```
[[], [1], [1, 2], [1, 2, 3], [1, 3], [2], [2, 3], [3]]
8
```

Subsets of [1,2,3]: at each element choose include / exclude → 2^n leaves



⚠ Interview trap

Adding `cur` itself instead of `new ArrayList<>(cur)` leaves every entry pointing to the same (finally empty) list.

Q303 TC-099 • Intermediate

Combination sum: find all combinations that add up to a target (reuse allowed).

⚡ Quick answer

Sort candidates; backtrack from index `start`, allowing the same index again (reuse), and stop early when a candidate exceeds the remaining target. Exponential in the worst case; pruning keeps it fast in practice.

© TechTribers. For personal use only. Do not redistribute.

```java
static List<List<Integer>> combinationSum(int[] candidates, int target) {
    int[] c = candidates.clone();
    Arrays.sort(c);
    List<List<Integer>> out = new ArrayList<>();
    dfs(c, target, 0, new ArrayDeque<>(), out);
    return out;
}
static void dfs(int[] c, int remain, int start, Deque<Integer> cur, List<List<Integer>> out) {
    if (remain == 0) { out.add(new ArrayList<>(cur)); return; }
    for (int i = start; i < c.length && c[i] <= remain; i++) {   // sorted: prune the rest
        cur.addLast(c[i]);
        dfs(c, remain - c[i], i, cur, out);                      // i, not i+1: reuse allowed
        cur.removeLast();
    }
}
System.out.println(combinationSum(new int[]{2, 3, 6, 7}, 7) + " " + combinationSum(new int[]{2, 3, 5}, 8));
```

▶ Output

```
[[2, 2, 3], [7]] [[2, 2, 2, 2], [2, 3, 3], [3, 5]]
```

Q304 TC-100 • Advanced

Solve the N-Queens problem.

⚡ Quick answer

Place one queen per row; for each column, check whether the column or either diagonal (`r - c` , `r + c`) is used, recurse to the next row, and undo on return. Track used columns and diagonals in boolean arrays for O(1) checks.

```java
static List<String> solutions = new ArrayList<>();
static int countQueens(int n) {
    solutions.clear();
    place(0, n, new int[n], new boolean[n], new boolean[2 * n], new boolean[2 * n]);
    return solutions.size();
}
static void place(int r, int n, int[] pos, boolean[] cols, boolean[] d1, boolean[] d2) {
    if (r == n) { solutions.add(Arrays.toString(pos)); return; }
    for (int c = 0; c < n; c++) {
        if (cols[c] || d1[r - c + n] || d2[r + c]) continue;
        pos[r] = c; cols[c] = d1[r - c + n] = d2[r + c] = true;
        place(r + 1, n, pos, cols, d1, d2);
        cols[c] = d1[r - c + n] = d2[r + c] = false;          // backtrack
    }
}
```

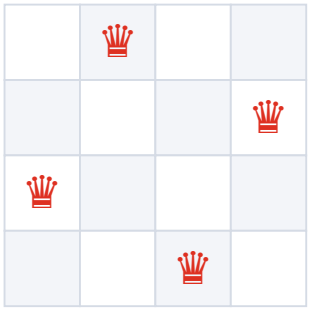

© TechTribers. For personal use only. Do not redistribute.

```
System.out.println("4 -> " + countQueens(4) + " " + solutions);
System.out.println("8 -> " + countQueens(8));
```

▶ Output

```
4 -> 2 [[1, 3, 0, 2], [2, 0, 3, 1]]
8 -> 92
```

4-Queens: place row by row, skip attacked columns/diagonals, backtrack on dead ends



`track used: cols, r - c, r + c`

4 → 2 solutions · 8 → 92

time ~O(n!) with pruning

💡 **Pro tip**

Each array entry `pos[r] = c` means the queen in row r sits in column c. Storing one column per row makes the "one queen per row" rule automatic.

© TechTribers. For personal use only. Do not redistribute.

Series progress

304 unique questions so far



Every question in the TechTribers Java series has one number, with duplicates removed across all parts. Part 7 starts at Q305.

Part	Questions	Count
Part 1	Q1–Q74	74
Part 2	Q75–Q121	47
Part 3	Q122–Q170	49
Part 4	Q171–Q204	34
Part 5	Q205–Q254	50
Part 6	Q255–Q304	50

More interview prep, carousels and roadmaps at **techtriber.com** and on Instagram **@techtribers**.

techtriber.com | Instagram @techtribers © TechTribers. For personal use only. Do not redistribute.